

Ownership & Borrowing — TOÀN BỘ qua HÌNH VẼ

Companion visual cho [ownership-borrowing.md](#). Mọi khái niệm được vẽ ra. Đọc tuần tự từ đầu — mỗi hình xây trên hình trước.

Mục lục

1. Bức tranh lớn: 3 trường phái quản lý bộ nhớ
2. Ownership = ai chịu trách nhiệm dọn?
3. Move — chuyển chủ sở hữu
4. Move vs Copy — so sánh
5. Clone — bản sao thật sự
6. Drop — hủy tự động
7. Borrow & — vay không sở hữu
8. &mut — vay độc quyền
9. Quy tắc borrowing
10. Bẫy: Vec realloc dangling
11. Reborrow
12. Lifetime — bản chất
13. Lifetime trong function signature
14. NLL — Non-Lexical Lifetimes
15. Two-phase borrows
16. Cell / RefCell
17. Mutex multi-thread
18. Box — single owner
19. Rc — đếm tham chiếu
20. Arc — atomic Rc
21. Weak — phá cycle
22. Rc<RefCell> pattern
23. Self-referential — vì sao cấm
24. Pin — neo cố định
25. Common errors trực quan
26. Bản đồ tư duy tổng

1. Bức tranh lớn

3 TRƯỜNG PHÁI QUẢN LÝ BỘ NHỚ

C/C++ — Lập trình viên TỰ làm

|| Bạn || malloc(); ✓

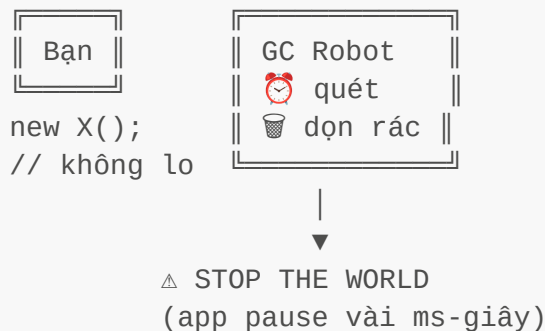
```

    free(); ..... ✓ ?
    free(); .... ✗ double-free
    use ptr; .... ✗ UAF

```

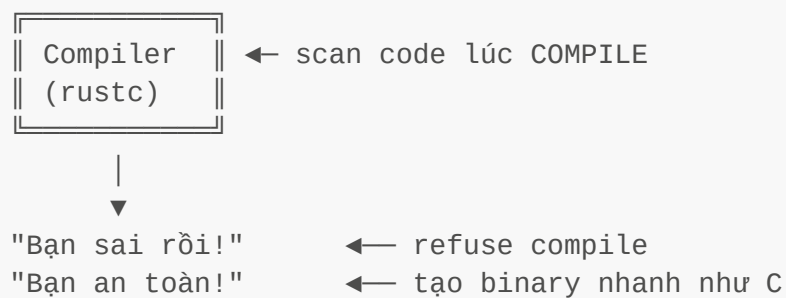
→ Nhanh, nhưng dễ sai

Java/Go/JS – GC làm thay



→ An toàn, nhưng chậm + tốn RAM

Rust – COMPILER kiểm tra giúp

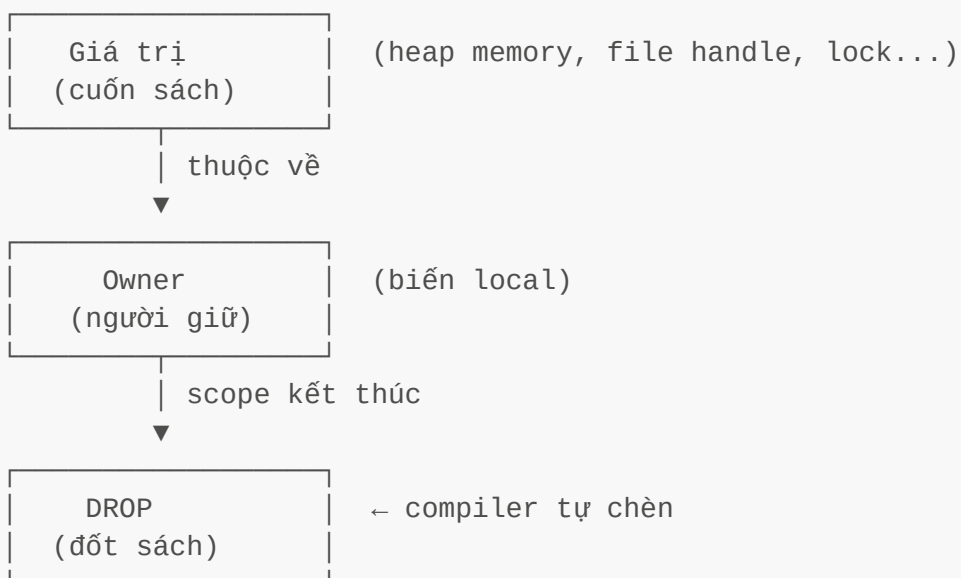


→ Nhanh như C + An toàn như Java + KHÔNG GC

2. Ownership là gì?

Định nghĩa siêu đơn giản:

Có cuốn SÁCH.
 Cần biết: AI giữ sách?
 Người đó là OWNER.
 Khi owner đi (out of scope), sách bị HỦY.



Code minh họa:

```
fn main() {
    let s = String::from("hi");    // 🖐️ s nhận sách
} // ————— s out scope → drop
```

3. Move

Bước 1: Trước khi gán

```
let s1 = String::from("hello");
```

STACK:

s1.ptr	→
s1.len = 5	
s1.cap = 5	

HEAP:

h	e	l	l	o
---	---	---	---	---

Bước 2: `let s2 = s1;`

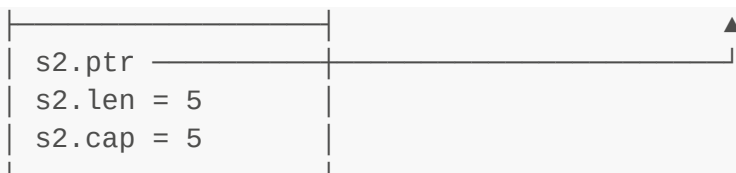
STACK:

s1: [INVALID]

← compile-time marker

HEAP:

h	e	l	l	o
---	---	---	---	---



← compiler ĐÁNH DẤU s1 không dùng được nữa
(không có lệnh runtime nào! chỉ là rule compile-time)

Bước 3: Cố dùng s1 → compile error

```
println!("{}", s1); // ❌ ERROR: use of moved value
```

❌ ERROR[E0382]

```

borrow of moved value: `s1`
--> 3:20
|
| let s2 = s1;
|           -- value moved here
| println!("{}", s1);
|                   ^^ value used
|                   after move

```

Vì sao move? Sơ đồ "nếu cho phép 2 owner"

GIẢ SỬ Rust CHO PHÉP cả s1 và s2:

STACK:



HEAP:



→ (cùng heap!)

Khi } đóng:

drop(s2) → FREE heap data ✓

drop(s1) → FREE heap data ❌

DOUBLE FREE!



Crash / corruption

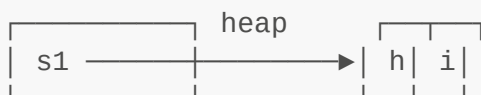
→ Rust **cấm** → chỉ 1 owner → 1 drop.

4. Move vs Copy

MOVE

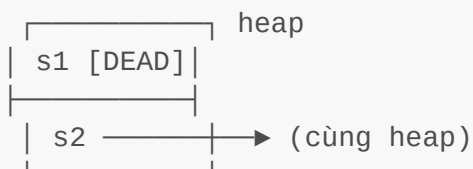
```
let s1 = String::from("hi");
let s2 = s1;
```

STACK trước:



chỉ 1 chủ

STACK sau:

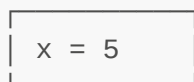


1 chủ chuyển sang

COPY

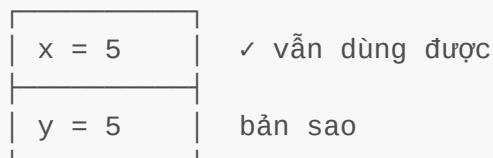
```
let x: i32 = 5;
let y = x;
```

STACK trước:



Không heap
→ copy bit là an toàn

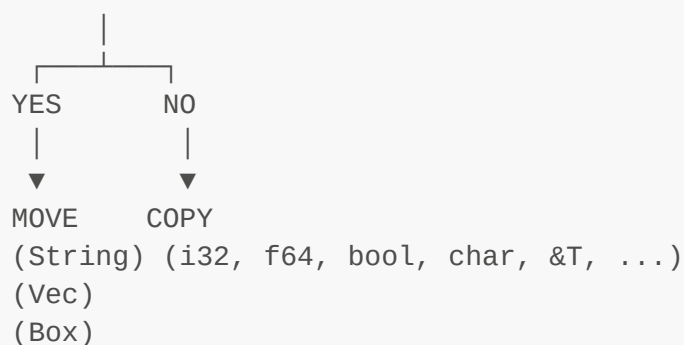
STACK sau:



Không heap
2 biến độc lập

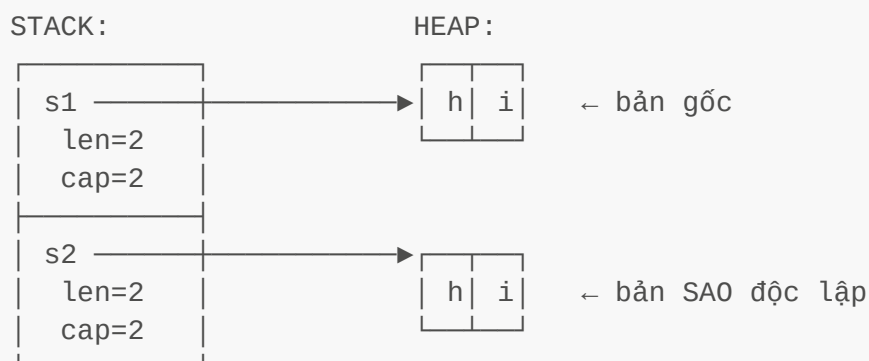
Quy tắc đơn giản

CÓ HEAP DATA?



5. Clone

```
let s1 = String::from("hi");
let s2 = s1.clone();
```



Chi phí:

```
let s2 = s1.clone();
```

- ├ 1. Yêu cầu allocator cấp heap mới (tốn)
- ├ 2. memcpy 2 byte từ heap cũ sang heap mới (tốn)
- └ 3. Copy struct (ptr, len, cap) vào s2

→ ĐẮT. Code chuyên nghiệp dùng `.clone()` có chủ đích.

6. Drop

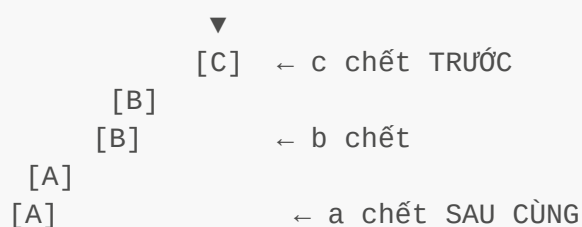
Drop tự động cuối scope

```
fn main() {
    let a = String::from("A");
    let b = String::from("B");
    let c = String::from("C");
    // ... code ...
} // ← TỰ ĐỘNG drop(c), drop(b), drop(a)
```

Thứ tự LIFO (ngược khai báo)

Khai báo:	a	b	c
	▼	▼	▼
Tạo:	[A]	[A]	[A]
		[B]	[B]
			[C]

Drop:



Vì sao LIFO?

```
let s = String::from("data");
let r = &s;           // r tham chiếu s
```

Nếu drop s TRƯỚC r:

r → (vùng nhớ rác) ❌ Dangling!

Nếu drop r TRƯỚC s: (LIFO - đúng!)

r chết → s vẫn sống bình thường ✓
sau đó s chết → free heap ✓

Sơ đồ Drop trait

```
struct Connection { name: String }

impl Drop for Connection {
    fn drop(&mut self) {
        println!("Đóng {}", self.name);
    }
}

fn main() {
    let c = Connection { name: "db".into() };
    // — drop(c) chạy → in "Đóng db"
```

Scope chính:

```
let c = Connection { ... };

... code ...

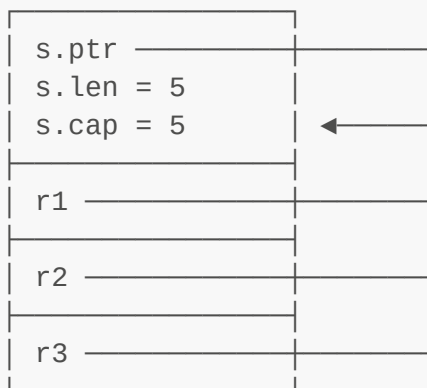
} ← chạm dấu này:
    1. compiler chèn drop(c)
    2. drop() in "Đóng db"
```

3. memory của c free

7. Borrow Immutable (&T)

```
let s = String::from("hello");
let r1 = &s;
let r2 = &s;
let r3 = &s;
```

STACK:



HEAP:



Tất cả trỏ vào STRUCT s
(không phải vào heap data!)

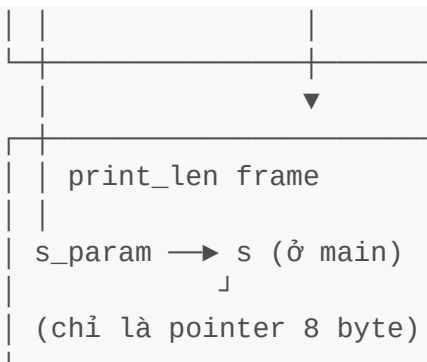
- Nhiều &T cùng OK
- Không ai sửa → an toàn
- Tất cả là pointer 8 byte (Copy được)

Pass &T sang hàm

```
fn print_len(s: &String) {
    println!("{}", s.len());
}

let s = String::from("hi");
print_len(&s);
println!("{}", s); // ✓ s vẫn sống
```





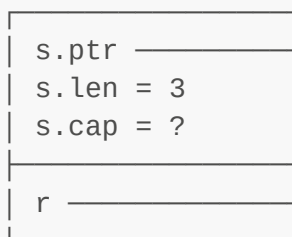
hàm return → frame pop

không ai động vào heap → s vẫn sống bên main ✓

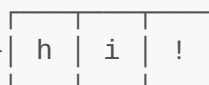
8. Borrow Mutable (&mut T)

```
let mut s = String::from("hi");
let r = &mut s;
r.push_str("!");
```

STACK:



HEAP:



← thêm '!'

CHỈ MỘT &mut, KHÔNG có &T nào khác

Cố tạo &mut thứ 2 → ERROR

```
let mut s = String::from("hi");
let r1 = &mut s;
let r2 = &mut s; // ✗
println!("{}", r1, r2);
```

✗ ERROR[E0499]
cannot borrow `s` as mutable more than once at a time

```
let r1 = &mut s;
----- first mut borrow
```

```
let r2 = &mut s;
      ^^^^^ second mut borrow!
```

Cố trộn &T và &mut T → ERROR

```
let r1 = &s;
let r2 = &mut s;           // ✗
println!("{}", r1, r2);
```

9. Quy tắc Borrowing

BORROW MATRIX

Có &mut T?	Có &T?	OK?
✗	✗	✓ (không ai mượn)
✗	✓	✓ (đọc shared)
✗	✓ NHIỀU	✓ (đọc shared)
✓ 1	✗	✓ (ghi độc quyền)
✓ 1	✓	✗ XUNG ĐỘT
✓ NHIỀU	*	✗ XUNG ĐỘT

Hình ảnh "thư viện sách"

SÁCH

 →  (đọc)
 →  (đọc)
 →  (đọc)

← &T (nhiều)

HOẶC

 →  (ghi vào)
 (không ai được vào)

← &mut T (1)

KHÔNG BAO GIỜ:

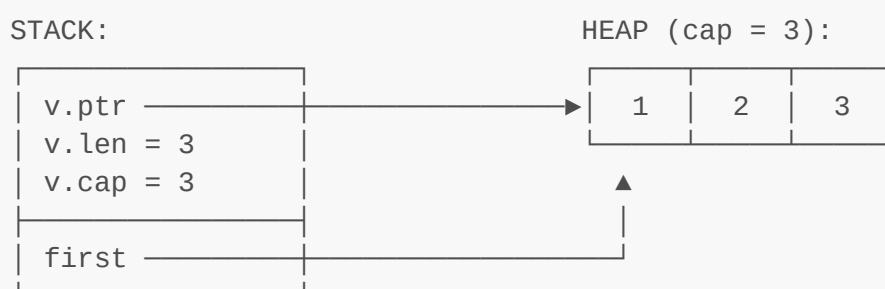
 (ghi) +  (đọc) cùng lúc

10. Vec Realloc — Bẫy Dangling

```
let mut v = vec![1, 2, 3];
let first = &v[0];           // borrow shared
v.push(4);                    // ❌ &mut conflict
println!("{}", first);
```

Lý do thực sự là gì?

Bước 1: trạng thái ban đầu



Bước 2: `v.push(4)` — `cap = 3` không đủ → REALLOC

Allocator cấp 1 block MỚI gấp đôi (`cap = 6`):

OLD HEAP (đang dùng):

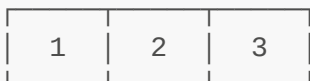


NEW HEAP (vừa cấp):



→ `memcpy [1,2,3]` sang block mới:

OLD HEAP (sắp free):



NEW HEAP:

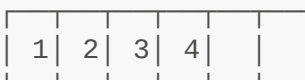


→ free OLD HEAP:

OLD HEAP (FREED):



NEW HEAP:



first VẪN trỏ vào đây

← DANGLING POINTER!

Bước 3: `println!("{}", first)` — đọc memory đã free!

→ Rust **cấm tại compile time** để không bao giờ xảy ra.

11. Reborrow

Bài toán

```
fn modify(s: &mut String) { s.push('!'); }

let mut s = String::from("hi");
let r = &mut s;
modify(r);           // r move vào? → r chết?
modify(r);           // sao vẫn dùng được?
```

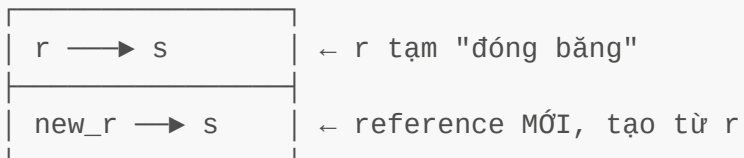
Sơ đồ reborrow

Trước `modify(r)`:



Khi gọi `modify(r)`:

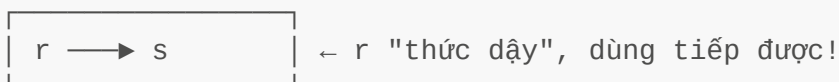
compiler ngầm chèn: `modify(&mut *r);`



↓
▼ pass vào `modify`



↓
▼ return → `new_r` drop



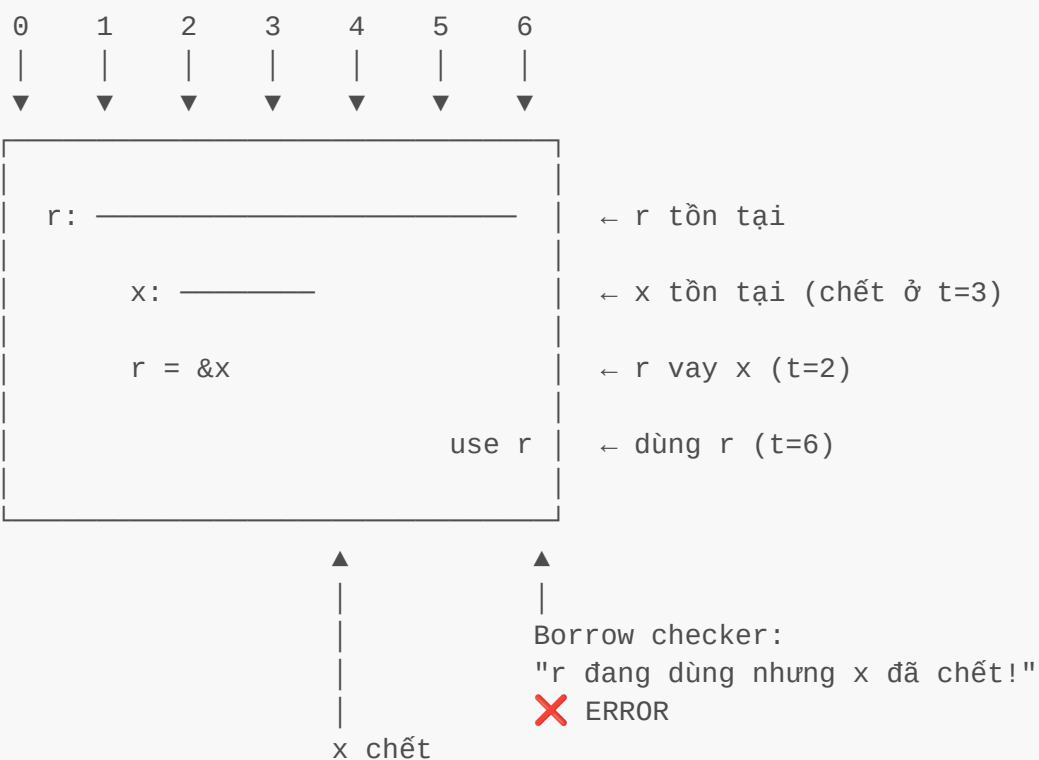
→ "Borrow của borrow" tạo reference mới, không move borrow cha.

12. Lifetime

Bản chất: "khoảng thời gian sống được"

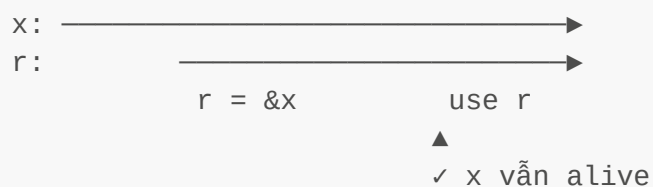
```
let r;
{
  let x = 5;
  r = &x;      // r mượn x
}              // x chết
println!("{}", r); // ❌
```

TIMELINE:



Sửa: đảm bảo data sống lâu hơn ref

```
let x = 5;      // x sống lâu
let r = &x;
println!("{}", r); // ✓ x vẫn sống
```



13. Lifetime trong signature

```
fn longest<'a>(s1: &'a str, s2: &'a str) -> &'a str {
    if s1.len() > s2.len() { s1 } else { s2 }
}
```

Đọc 'a như thế nào?

```
fn longest<'a>(s1: &'a str, s2: &'a str) -> &'a str
```

Tên hàm

Khai báo lifetime parameter

Hai input dùng CHUNG lifetime 'a

Input 1: sống 'a

Input 2: sống 'a

Trả về &str sống 'a

Output sống 'a

Bản chất 'a là gì?

'a = INTERSECTION (giao) của lifetime caller pass vào

[illegible]

t: 0 1 2 3 4 5 6 7 8 9 10
s1: —————→
s2: —————→ (chết t=5)
r: —————→ (tối đa t=5)

dùng r ở đây là OK
dùng r ở $t=7 \rightarrow \text{vì } s_2 \text{ (nguồn) chết}$

14. NLL — Non-Lexical Lifetimes

Trước NLL: lifetime = lexical scope `{ }`

```
let mut v = vec![1, 2, 3];
let first = &v[0];
println!("{}", first);
v.push(4); // ✗ trước NLL
```

Lifetime first (cũ – lexical):

```
let first = &v[0];
println!("{}", first);
v.push(4);
}
```

———— sống đến `}`

———— conflict với first!

▲
Trước NLL: first vẫn alive ở đây → ERROR

Sau NLL: alive đến LAST USE

```
Lifetime first (NLL):

let first = &v[0];
println!("{}", first);
v.push(4);
}
```

first alive
← LAST USE

↑
first CHẾT ngay sau dòng này

✓ OK vì first đã chết

Sơ đồ control flow



↓
▼ first CHẾT (NLL)

v.push(4)

← &mut v được phép ở đây

15. Two-phase borrows

```
let mut v = vec![1, 2, 3];
v.push(v.len()); // ✓ compile được – nhờ two-phase
```

Diễn giải: 2 pha của &mut

v.push(v.len())

↑ ↑
| |
└─ cần &mut self để gọi .push()
└─ cần &v để gọi .len()

THỜI ĐIỂM 1: Evaluate arguments

Compiler bắt đầu CHUẨN BỊ &mut v ("reserved" phase)
nhưng chưa "active" – vẫn cho phép &v khác

v (mut ref reserved)

↓

▼

&v.len() → return 3

← cùng tồn tại OK!

THỜI ĐIỂM 2: Run the function

Mọi &v đã chết. &mut v "activate" thật sự.

v.push(3)

↓

▼

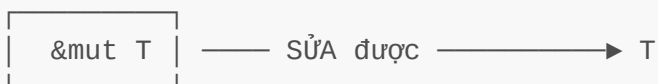
&mut v ACTIVE – push 3

→ Đây là lý do bạn dùng code hợp lý nhưng **không bao giờ phải nghĩ về 2 phase** — compiler tự lo.

16. Cell / RefCell

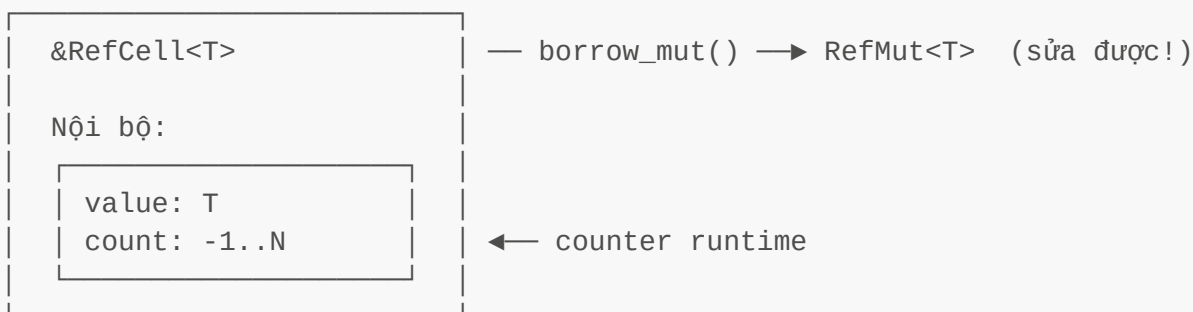
Bài toán: mutate qua &T

Quy tắc thông thường:



Nhưng đôi khi: chỉ có &T, vẫn cần sửa!

RefCell — vỏ bọc cho phép



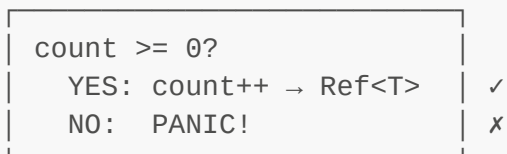
RefCell hoạt động

Trạng thái counter:

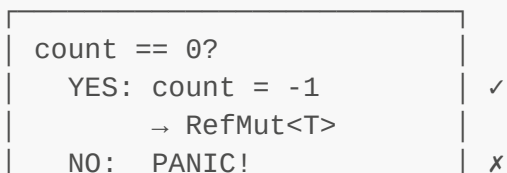
```

count = 0      ← không ai mượn (rảnh)
count = 3      ← 3 người mượn &T (read)
count = -1     ← 1 người mượn &mut T (write)
  
```

Khi gọi borrow():



Khi gọi borrow_mut():



Khi Ref/RefMut drop:

Khôi phục counter
(++ nếu là RefMut,
-- nếu là Ref)

Khác biệt: compile-time vs runtime

&T / &mut T:

Check tại COMPILE
Lỗi → không compile
Zero cost runtime
Phân tích static

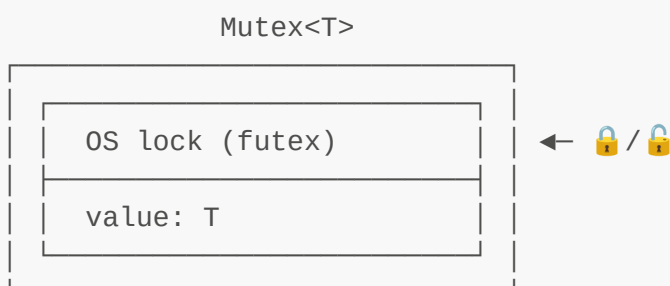
RefCell:

Check tại RUNTIME
Lỗi → panic
Có cost (++/-- counter)
Linh hoạt hơn

17. Mutex multi-thread

```
let m = Mutex::new(0);
{
    let mut g = m.lock().unwrap();
    *g += 1;
} // ← g drop → unlock
```

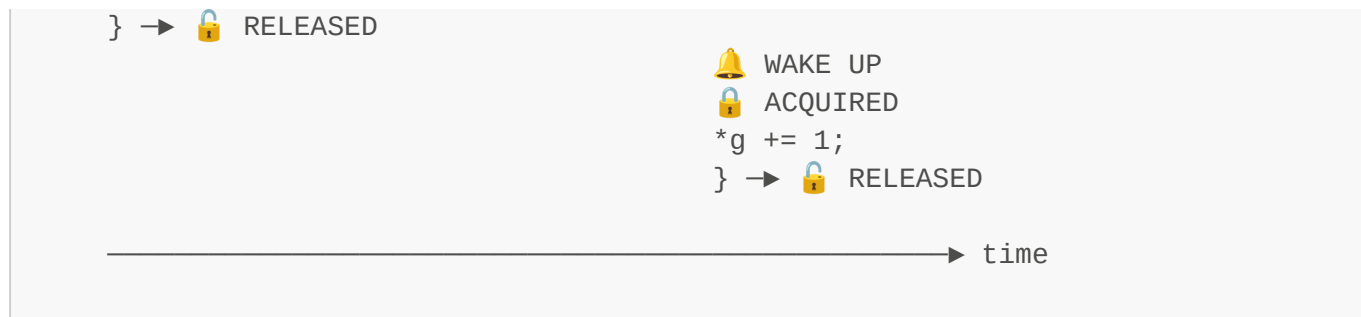
Sơ đồ Mutex



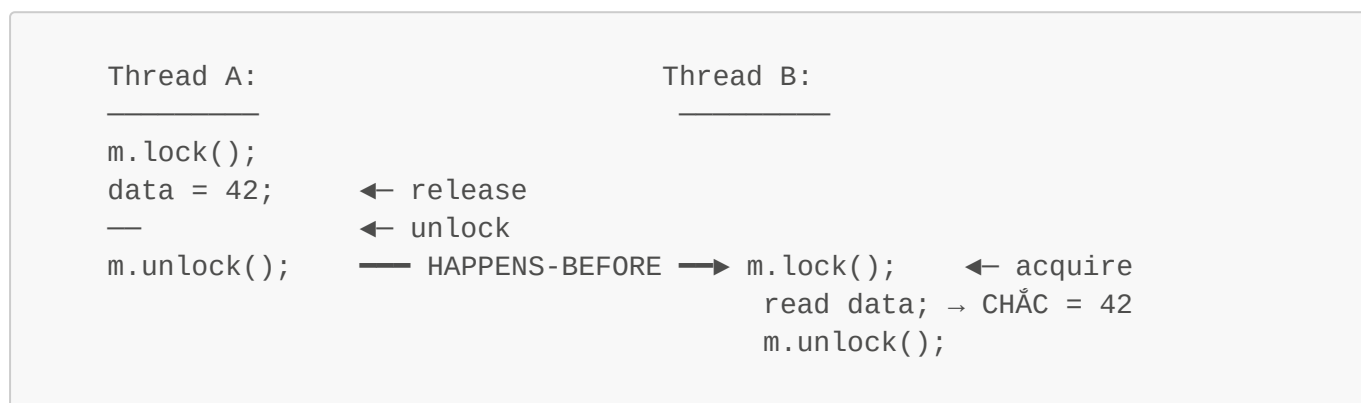
Khi 2 thread tranh chấp

Thread A
m.lock() → 🔒 ACQUIRED
*g += 1;

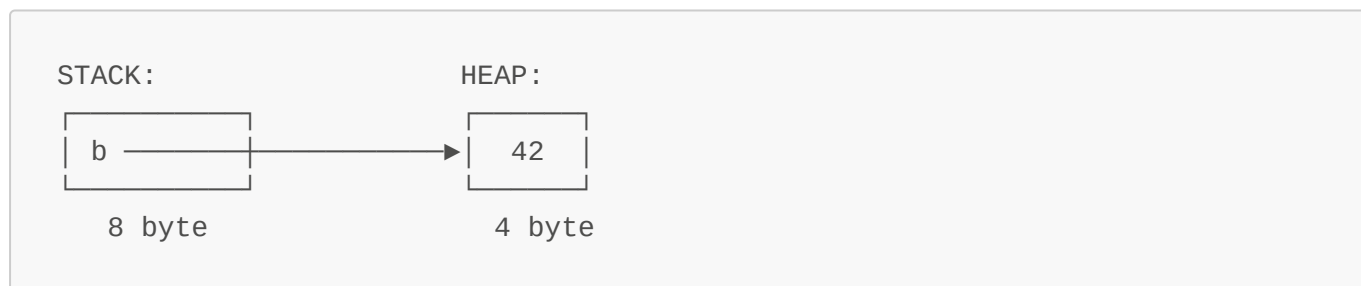
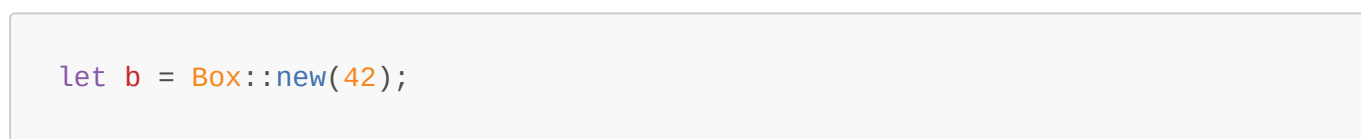
Thread B
m.lock() → || BLOCKED
(kernel sleep)



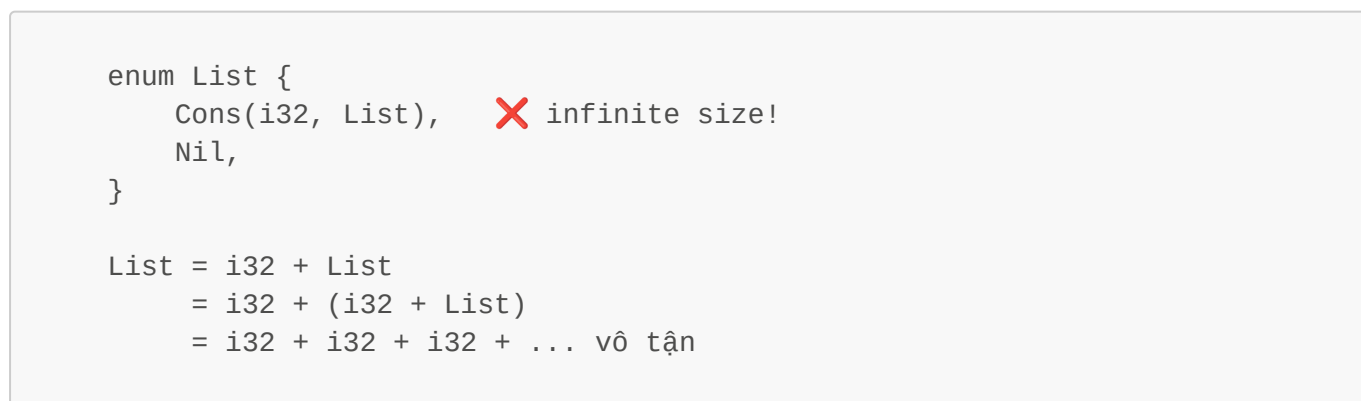
Memory ordering tự động



18. Box — single owner trên heap

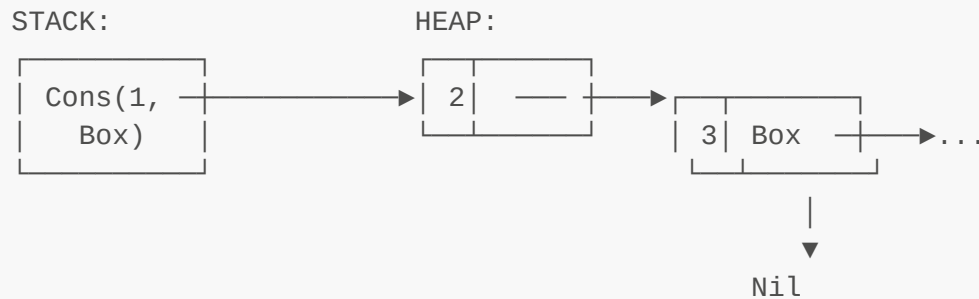


Box giải quyết: recursive types



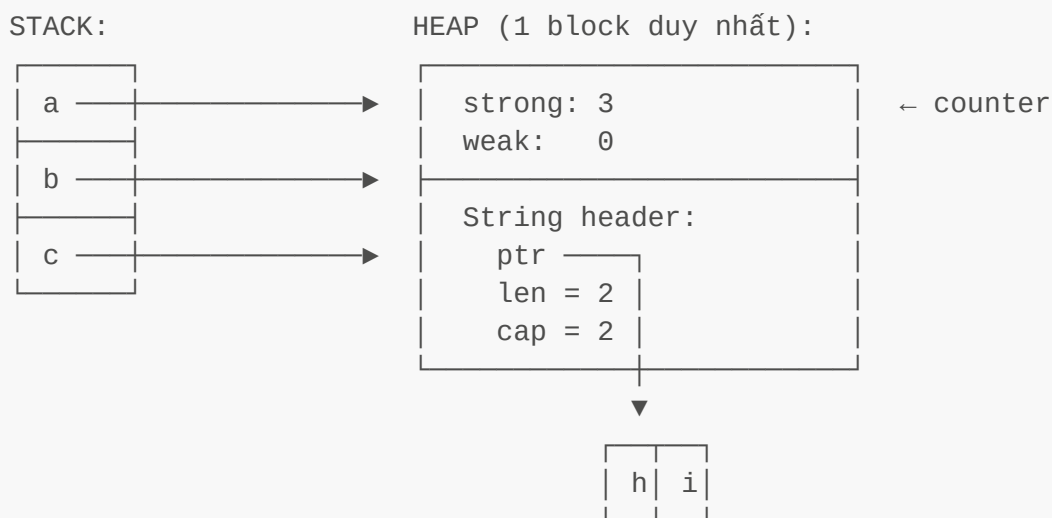
Sửa: Box

```
enum List {
  Cons(i32, Box<List>),  ✓ Box = 8 byte cố định
  Nil,
}
```



19. Rc — Reference Counted

```
let a = Rc::new(String::from("hi"));
let b = Rc::clone(&a);
let c = Rc::clone(&a);
```



Diễn biến strong count

Bước 1: `let a = Rc::new(...)`

`strong = 1, weak = 0`

Bước 2: `let b = Rc::clone(&a)`

strong = 2, weak = 0

❶ KHÔNG clone string, chỉ ++ counter (cheap)

Bước 3: let c = Rc::clone(&a)

strong = 3, weak = 0

Bước 4: c hết scope → drop(c)

strong = 2, weak = 0

❷ heap data CHƯA free vì còn a, b

Bước 5: b hết scope → drop(b)

strong = 1

Bước 6: a hết scope → drop(a)

strong = 0 → 🗑️ FREE toàn bộ heap (header + string data)

Cảnh báo: Rc không multi-thread!

Rc<T> dùng count thường (i32, không atomic).

Hai thread cùng clone:

Thread A: count = 5

⋮ load 5

Thread B: load 5

Thread A: add 1 → 6

⋮ write 6

Thread B: add 1 → 6

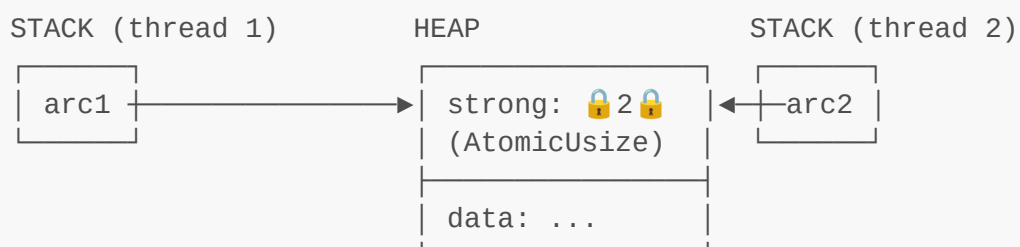
write 6

→ count = 6 (đáng lẽ phải = 7!)

→ LEAK 1 reference → bug!

→ Cần Arc cho multi-thread.

20. Arc — Atomic Rc



▲
|
atomic_increment
(LOCK XADD trên x86)

So sánh Rc vs Arc

	<u>Rc<T></u>	<u>Arc<T></u>
Counter:	i32	AtomicUsize
Clone cost:	++ (~1 ns)	LOCK XADD (~5 ns)
Thread:	✗ single-thread	✓ multi-thread
Sync:	✗	✓
Send:	✗	✓

Khi nào dùng?

- Single-thread: dùng Rc (nhanh hơn)
- Multi-thread: dùng Arc (an toàn)

21. Weak — phá cycle

Vấn đề: Rc cycle = memory leak

```
struct Node {
    children: Vec<Rc<Node>>,
    parent: Rc<Node>,           // ✗ cycle
}
```

Khởi tạo:

```
root → Node A {
    parent: Node R,           ← R giữ A
    children: [Node B]       ← A giữ B
}
Node B {
    parent: Node A,           ← A giữ B, B giữ A – cycle!
    children: []
}
```

Strong counts:

- A.strong = 2 (root + B.parent)
- B.strong = 1 (A.children)

Khi root drop:

A.strong-- = 1 (vẫn còn B.parent giữ A)

- ① A KHÔNG bị free
- ① A.children vẫn giữ B
- ① B vẫn giữ A.parent (chính là A!)

```
A.strong = 1, B.strong = 1
→ KHÔNG BAO GIỜ về 0
→ LEAK 100% memory
```

Giải pháp Weak

```
parent: Weak<Node>    ← weak không tăng strong!
```

Strong counts (sau khi drop root):

- A.strong = 1 (chỉ root cũ – đã drop) = 0 ✓ FREE A!
- Khi A free → A.children drop → B.strong = 0 → FREE B

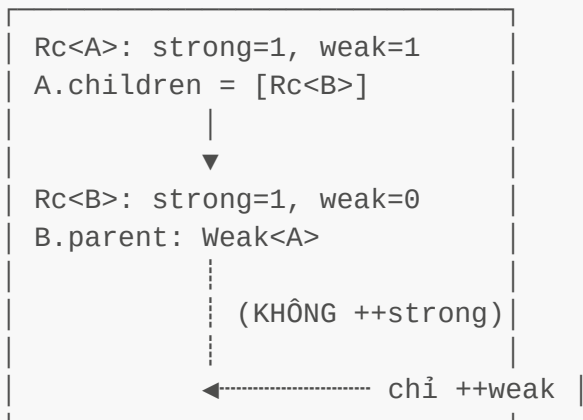
Weak counts được dùng để track:

"có ai đang muốn 'upgrade' weak thành strong không?"

Sơ đồ Weak

```
parent: Weak<Node>
      |
      | .upgrade()
      ▼
Option<Rc<Node>>
  |      |
Some(rc) None    ← parent đã chết!
(vẫn    (an toàn,
sống)   không crash)
```

STACK / HEAP:



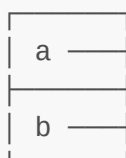
22. Rc<RefCell> pattern

Bài toán: Rc cho shared owner, RefCell cho mutate qua &T

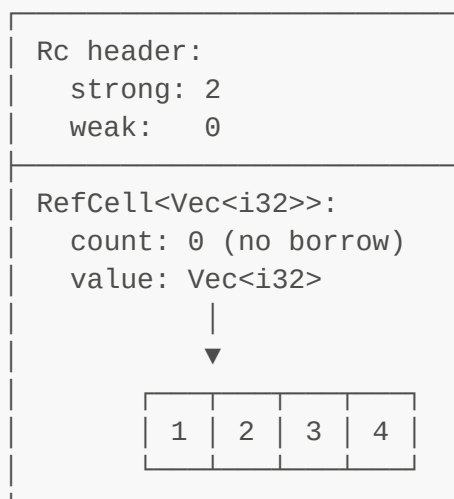
```

let a = Rc::new(RefCell::new(vec![1, 2, 3]));
let b = Rc::clone(&a);
a.borrow_mut().push(4); // ✓
println!("{}", b.borrow()); // [1,2,3,4]
  
```

STACK:



HEAP:



Layer hoạt động



RefCell<...> → mutation qua &T
(count runtime -1..N)

Vec<i32> → data thực sự

Phiên bản multi-thread

Arc<Mutex<T>>:

Arc<...> → shared atomic ownership

Mutex<...> → mutex lock OS

T (data thực)

23. Self-referential — vì sao cấm

```
struct Bad {
    data: String,
    ptr: &str,      // trỏ vào data của chính mình
}
```

Tại sao nguy hiểm?

Bước 1: Bad ở địa chỉ 0x1000

STACK 0x1000:

data.ptr
data.len = 2
data.cap = 2

HEAP:

h	i
---	---



```
ptr _____
```

Bước 2: Move Bad sang 0x2000

STACK 0x1000:

```
[empty, đã move]
```

HEAP:

```
[ h | i ]
```

STACK 0x2000:

```
data.ptr _____
data.len = 2
data.cap = 2

ptr _____
```

← data vẫn OK (chứa ptr riêng)

→ (địa chỉ cũ 0x1000)

✗ DANGLING!
ptr vẫn lưu địa chỉ TUYỆT ĐỐI
của vùng cũ

→ Move trong Rust = memcpy. ptr **không** được tự động cập nhật → dangling.

→ Rust **cấm self-reference trực tiếp**.

24. Pin — neo cố định

Pin = "cấm move"

Pin<&mut T>:

```
Wrapper (chỉ marker type)
  |
  | Pointer trong
  |
```

API: bạn KHÔNG được:

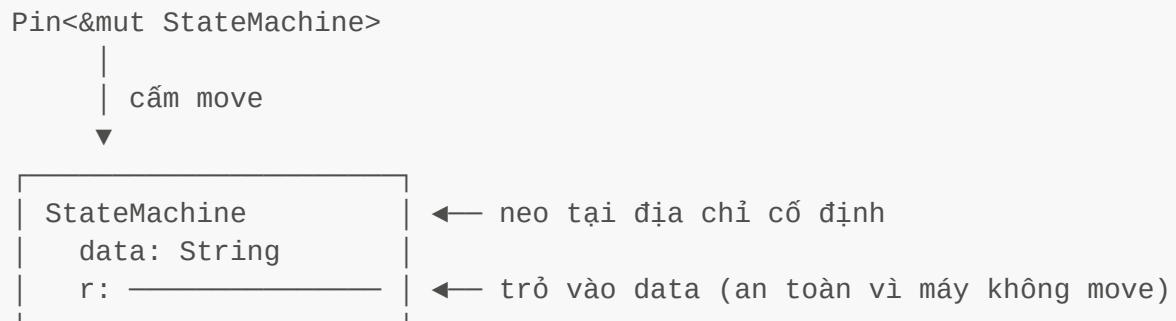
- swap(&mut *p, ...)
- replace
- mem::take

→ tức là không di chuyển bất cứ thứ gì

```
async fn foo() {
    let data = String::from("hi");
    let r = &data;           // r tham chiếu data
    yield_point.await;
    println!("{}", r);
}
```

```
StateMachine {
    data: String,
    r: &data,           ← self-reference!
    state: AfterAwait,
}
```

Pin giải quyết



→ **Bạn không bao giờ phải dùng Pin** trừ khi viết async runtime hoặc unsafe code.

```
let s = String::from("hi");
let s2 = s;
println!("{}", s);
      ^
```

△ ERROR

s →moved→ s2

```
println!("{}", s);
           ^ Đã chết!
```

Fix: clone, hoặc borrow

```
let s2 = s.clone();    // hoặc
let s2 = &s;
```

Error 2: borrow + mut conflict

```
let mut v = vec![1,2,3];
let r = &v[0];
v.push(4);
println!("{}", r);
```

r → v[0]

|
v.push(4) - &mut v xung đột với r

✗ ERROR[E0502]

Fix: dùng r xong rồi mới push

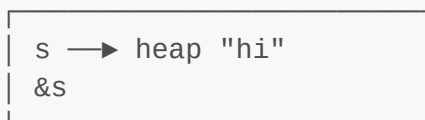
```
let r = &v[0];
println!("{}", r); // r last use
v.push(4);         // ✓ OK
```

Error 3: dangling reference

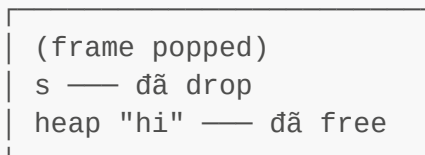
```
fn make() -> &String {
    let s = String::from("hi");
    &s
}
```

Stack timeline:

make() call:



make() return:



return &s — dangling!

Fix: return String, không return &String

```

fn make() -> String {
    String::from("hi")
}
  
```

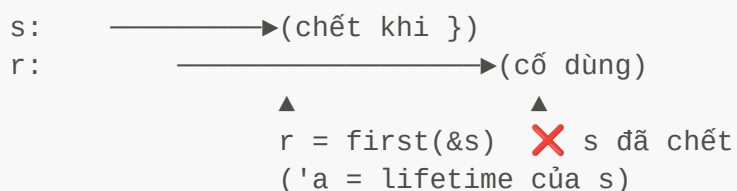
Error 4: lifetime mismatch

```

fn first<'a>(s: &'a str) -> &'a str { ... }

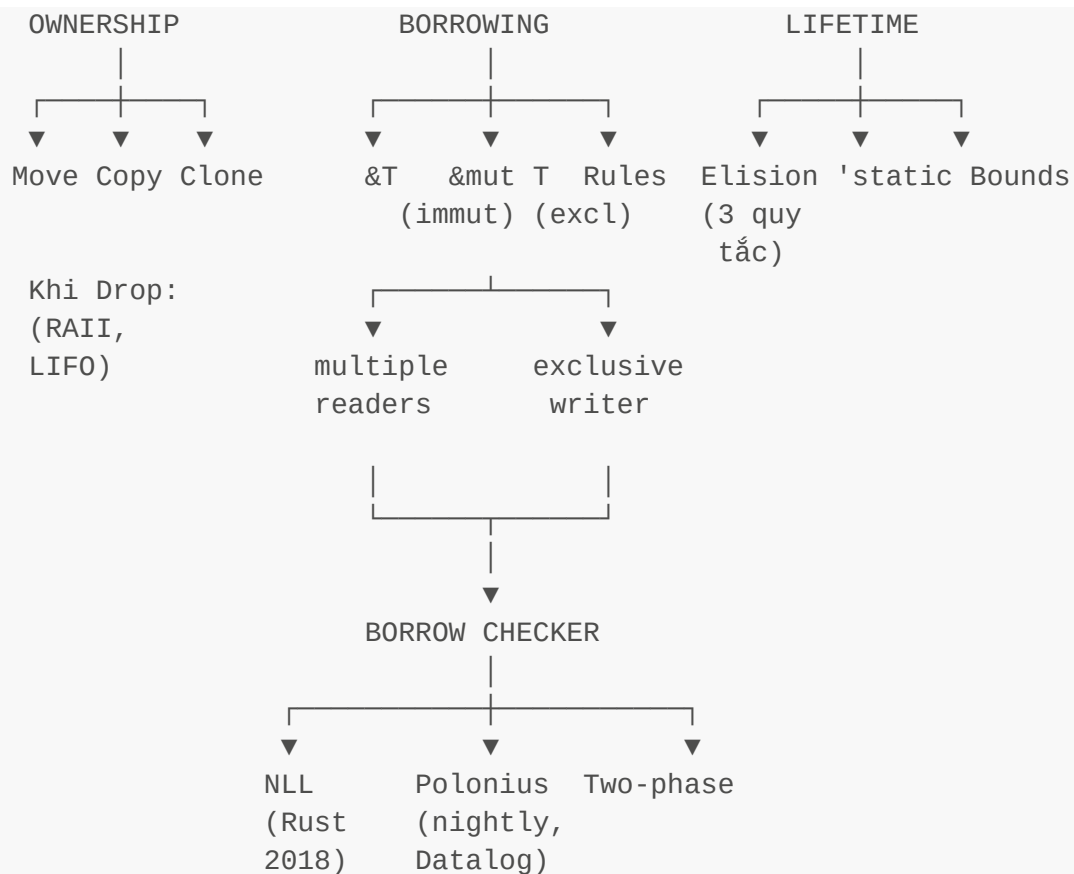
let r;
{
    let s = String::from("hi");
    r = first(&s);    // r mượn từ s ('a = scope của s)
}                  // s chết
println!("{}", r);  // ❌ r dangling
  
```

Visual:



26. Mind Map — Bản đồ tư duy tổng





INTERIOR MUTABILITY (mutate qua &T – runtime check)

Single-thread:

- Cell<T>
- RefCell<T>

Multi-thread:

- Mutex<T>
- RwLock<T>
- Atomic*

ALL BUILT ON: UnsafeCell<T>

SHARED OWNERSHIP

- Box<T> → 1 owner duy nhất trên heap
- Rc<T> → nhiều owner, count thường (single-thread)
- Arc<T> → nhiều owner, count atomic (multi-thread)
- Weak<T> → tham chiếu KHÔNG giữ sống (phá cycle)

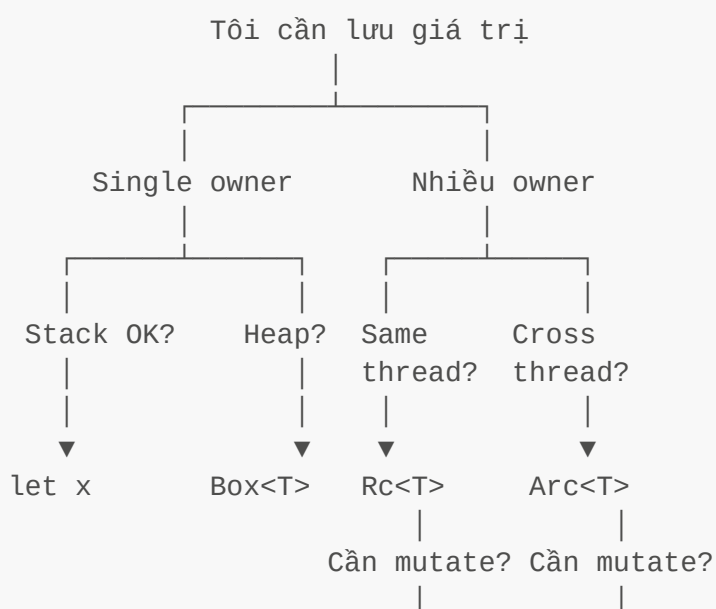
PATTERNS NÂNG CAO

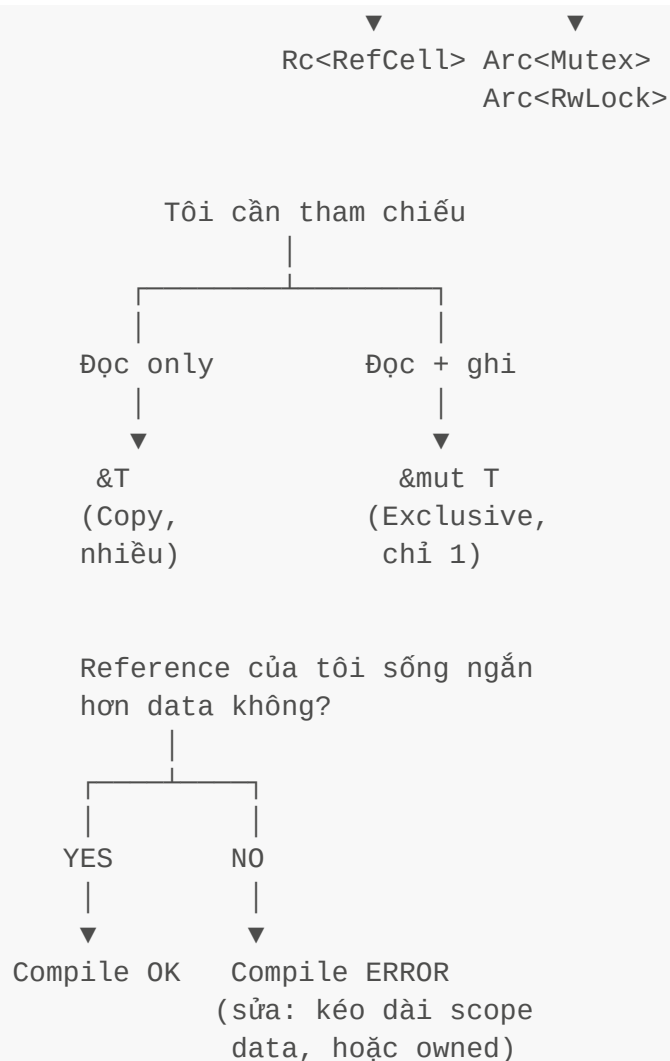
<code>Rc<RefCell<T>></code>	single-thread shared+mutable
<code>Arc<Mutex<T>></code>	multi-thread shared+mutable
<code>Pin<P></code>	cấm move (cho self-ref / async)
<code>PhantomData<T></code>	lifetime ảo (zero-size)

Tổng kết — 7 Quy luật Vàng

1. Mỗi giá trị có ĐÚNG 1 owner
2. Khi owner đi → giá trị bị drop
3. Move chuyển owner, KHÔNG copy heap
4. `&T`: nhiều cùng OK, nhưng không cùng `&mut T`
5. `&mut T`: chỉ MỘT tại 1 thời điểm
6. Reference KHÔNG sống lâu hơn dữ liệu nó vay
7. Cần "mutate qua `&T`" → dùng interior mutability

Bonus: Sơ đồ "đi đâu khi gặp bài toán"





Lời cuối: Mọi quy tắc Rust đều **mô hình hoá** một quy luật của memory thực sự. Quy tắc không phải "phiên phức" — đó là cách compiler giúp bạn không bao giờ gặp các bug C/C++ đã ám ảnh suốt 50 năm.

Đọc kèm [ownership-borrowing.md](#) (lý thuyết) và [memory-model-visual.md](#) (bộ nhớ vật lý) để hiểu trọn vẹn.